

윤재현

Jaehyun Yoon · Full-stack Developer

반복은 구조로, 모호함은 기준으로 바꿉니다.

같은 문제를 매번 새로 풀지 않기 위해, 반복되는 것은 한 번 제대로 풀어 구조로 남깁니다. 매번 새로 풀면 같은 코드가 여기저기 쌓이고, 하나를 고치려면 전부 열어봐야 합니다. 모호한 요구는 측정할 수 있는 기준으로 바꾼 뒤에야 설계를 시작합니다. "빠르게"라는 말은 아무것도 결정해 주지 않습니다. 문제가 막히면 우회하지 않고 내부가 실제로 어떻게 동작하는지 끝까지 파고듭니다. 겉으로 동작하는 코드는 예외 상황에서 반드시 흔들립니다.

PROJECTS

convene	화상회의·AI 회의록 플랫폼 (1인 풀스택) NestJS · Mediasoup · Redis · MongoDB · AWS	2026.05 - 현재
express-cargo	Express 요청 바인딩 라이브러리 (오픈소스·5인) TypeScript · reflect-metadata	2025.06 - 현재
Plum	실시간 화상강의 플랫폼 (백엔드·4인) NestJS · Mediasoup · Redis · Faster-whisper	2025.12 - 2026.02
Fienmee	행사·축제 커뮤니티 플랫폼 (풀스택·7인) Express · MongoDB · Next.js · React Native	2024.08 - 2025.12
Doculink	Confluence 문서 관계 그래프 확장 (풀스택·4인) Forge · Three.js · retext-keywords	2024.12 - 2025.06

PROBLEM-SOLVING CASES

01	트러블슈팅	AI가 생성한 코드가 코드베이스와 어긋나는 문제	convene
02	설계	실패해도 유실되지 않는 회의록 파이프라인	convene
03	설계	단일 계층뿐이던 바인딩을 무한 중첩까지	express-cargo
04	설계	잘못된 데코레이터 조합을 시작 시점에 차단	express-cargo
05	설계	Redis ZSET으로 설계한 실시간 참여도 랭킹	Plum
06	설계	3분 내 요약·보안·CPU 제약을 동시에 만족하는 STT	Plum
07	트러블슈팅	Redis Pub/Sub 소켓 이벤트 순서 보장	Plum
08	트러블슈팅	끊김과 이탈을 구분할 수 없는 재접속 문제	Plum
09	설계	흩어진 요청 검증을 라이브러리로 표준화	Fienmee
10	트러블슈팅	하나의 잘못된 데이터가 전체 수집을 막는 문제	Fienmee
11	설계	던지고 끝나지 않는 푸시 알림 시스템 설계	Fienmee
12	트러블슈팅	Windows에서만 실패하는 안드로이드 빌드	Fienmee
13	설계	플랫폼 제한 안에서 정확한 키워드 추출	Doculink

AI가 생성한 코드가 코드베이스와 어긋나는 문제

Problem

혼자 빠르게 출시하기 위해 AI 코딩 에이전트로 Plum → convene 마이그레이션을 진행했다. 그런데 AI가 생성한 코드가 개별로는 동작하면서도 기존 코드베이스의 컨벤션·구조와 미묘하게 어긋나는 일이 반복됐다. 그대로 두면 코드베이스 전체의 일관성이 무너져 유지보수가 불가능해질 상황이었고, "그렇듯하게 동작하는 코드"와 "옳은 코드"를 가르는 기준이 필요했다.

Solving

1 — 판단 기준을 규약 문서로 먼저 고정

WHY AI는 옳은 기준을 스스로 세우지 못함 · HOW CLAUDE.md · ARCHITECTURE.md

도메인 모델·의존성 규칙·커밋 규약을 문서로 먼저 정의하고, AI는 이 규약안에서만 코드를 생성하게 했다. 무엇이 옳은 구조인지에 대한 설계 판단은 직접 내리고, 반복적인 구현만 AI로 가속하는 역할 분담을 명확히 했다.

2 — 기존 코드 참고 강제 + TDD로 동작을 먼저 고정

WHY 규약만으로는 세부 컨벤션 이탈이 남음 · HOW 참고 강제 · 테스트 우선 구현

코드를 작성할 때 반드시 기존 코드베이스를 먼저 참고하도록 규약에 못 박고, 기능마다 테스트를 먼저 정의한 뒤 구현하게 했다. 의도한 동작이 테스트로 고정되니, AI가 만든 코드도 같은 기준선 안으로 일관되게 수렴했다.

Flow



Result

- AI의 개발 속도와 일관된 코드 품질을 동시에 확보해 혼자서 모든 작업을 진행
- 규약 문서가 곧 온보딩 문서가 되어, 팀 확장 시 새 합류자도 같은 기준으로 작업 가능

실패해도 유실되지 않는 회의록 파이프라인

Problem

회의가 끝나면 음성을 STT → 요약 두 단계로 처리해 회의록을 만든다. 그런데 host가 종료하거나 전원이 나가는 등 종료 시점이 제각각 이고, STT나 요약이 도중에 실패할 수 있었다. 실패했다고 회의록 자체가 사라지면 사용자는 회의 기록을 통째로 잃는다. **"언제 생성을 시작하고, 실패해도 무엇이 반드시 남길 것인가"**를 상태 기계로 명확히 정의해야 했다.

Solving

1 — idle 조건으로 생성 시작 시점을 일원화

WHY 종료 경로가 여러 가지 · HOW Live · Idle · Ending 상태 분리

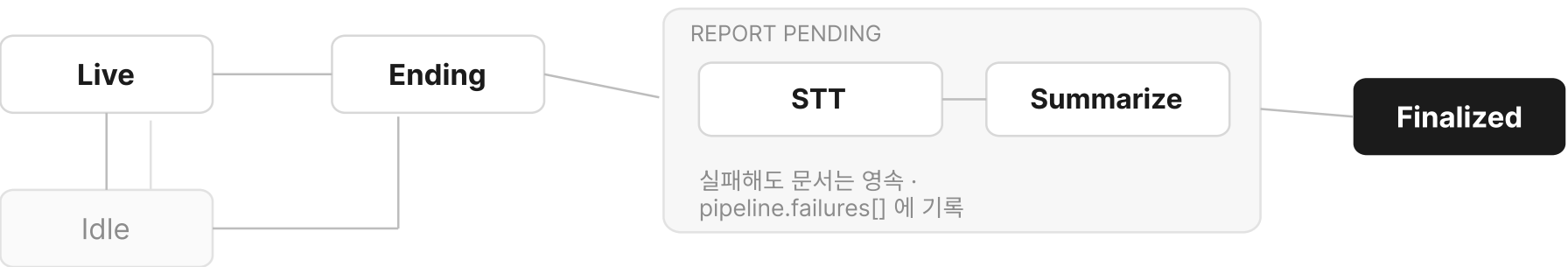
host 종료든 전원 퇴장이든, 회의가 idle 상태로 일정 시간 지속되면 Ending을 거쳐 회의록 생성을 시작하도록 진입 조건을 하나로 모았다. 어떤 경로로 끝나도 같은 지점으로 수렴한다.

2 — 문서 영속을 먼저, 실패는 기록으로 격리

WHY 부분 실패에도 데이터 보존 · HOW pipeline.failures[] 기록 후 진행

STT·요약을 REPORT PENDING 안의 하위 단계로 두고, 각 단계가 실패해도 문서는 영속시킨 채 실패 원인을 pipeline에 기록하도록 했다. 두 단계가 모두 끝나야 Finalized로 확정되며, 실패해도 재요약으로 복구할 수 있다.

State



Result

- 종료 경로와 무관하게 회의록 생성이 일관되게 시작되고, 부분 실패에도 문서는 보존
- 실패 원인이 기록으로 남아 재요약으로 복구 가능 — 사용자는 회의 기록을 잃지 않음

단일 계층뿐이던 바인딩을 무한 중첩까지

Problem

초기 바인딩 로직은 단일 계층 속성만 처리할 수 있어 객체 안의 객체나 복합 타입을 다루지 못했고, 변환과 검증 로직이 엔진 내부에 뒤섞여 기능을 확장할 때마다 핵심 로직을 건드려야 했다. 중첩 구조에서 검증이 실패해도 어디가 잘못됐는지 알 수 없었다.

Solving

1 — 재귀적 바인딩 파이프라인 재설계

WHY 무한 계층 중첩 객체 대응 · **HOW** 속성이 객체면 자신을 재귀 호출

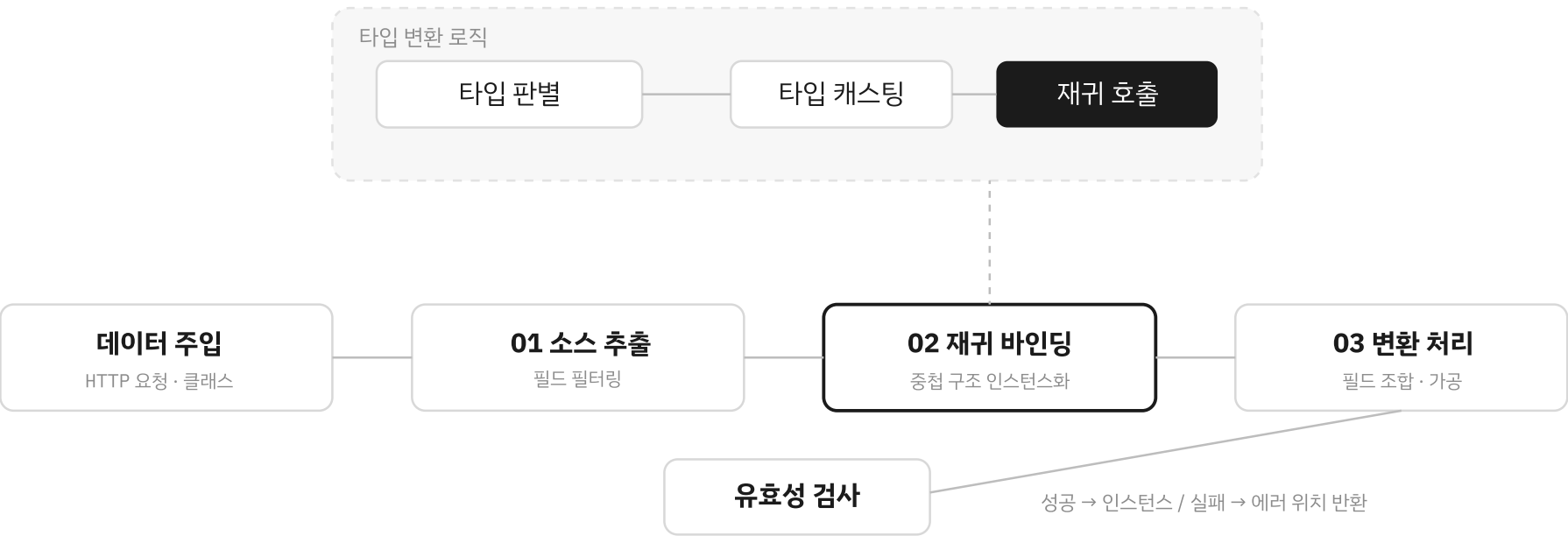
단일 계층만 처리하던 로직을 모듈화해, 속성이 객체면 스스로를 호출하는 재귀 파이프라인으로 재설계했다. 상위 경로를 스택처럼 함께 전달해 깊은 계층의 오류 위치까지 정확히 반환한다.

2 — 관심사 분리 · 다형성으로 확장 기틀 마련

WHY 확장 시 핵심 로직 수정 위험 제거 · **HOW** 변환·바인딩·검증 독립 모듈 + Discriminator · Lazy Evaluation

엔진에 혼재된 로직을 변환 → 바인딩 → 검증 독립 모듈로 분리해 유니언 타입·Enum을 핵심 로직 수정 없이 결합할 수 있게 했다. 런타임에 타입을 결정하는 Discriminator를 도입하고 Lazy Evaluation으로 중첩 객체의 순환 참조를 해결했다.

Architecture — 재귀 바인딩 파이프라인



Result

- 데코레이터 선언만으로 중첩 객체 전체를 자동 바인딩·검증, 깊은 계층의 에러 위치까지 정밀 반환 · 유니온 타입·복합 배열 지원
- 실제 서비스(Fienmee)에 적용해 컨트롤러의 검증 책임을 payload 계층으로 분리하며 재사용성 검증

잘못된 데코레이터 조합을 시작 시점에 차단

Problem

데코레이터가 20여 종으로 늘면서, Source 중복 지정·타입과 안 맞는 Validator·존재하지 않는 필드 참조 같은 잘못된 조합이 런타임 첫 요청에야 드러나는 문제가 생겼다. **배포 후에야 터지는 침묵 버그**였고, 그렇다고 모든 조합을 무작정 막으면 개발 편의성이 떨어졌다.

Solving

1 — 데코레이터를 5개 카테고리로 정리하고 규칙화

WHY 규칙을 세우려면 분류가 먼저 · **HOW** Source·Request·Virtual·Transform·Validator

모든 데코레이터를 5개 카테고리로 정리하고, 모든 필드는 반드시 출처(Source/Request/Virtual) 하나를 가져야 한다는 기준을 세웠다. 오용은 심각도(P0~P2)로 차등해, 카테고리 충돌 같은 명백한 오류와 존재하지 않는 필드 참조 같은 침묵 버그를 우선 차단하고 컴파일러가 이미 막는 경우는 검증에서 제외했다.

2 — 분석과 검증을 분리, 시작 시점 1회 분석 후 재사용

WHY 매 요청 재분석 낭비 제거 · **HOW** bindingCargo 시점 분석 · 메타데이터 캐시

클래스 분석과 검증 로직을 분리해, 서버 시작(bindingCargo 호출) 시점에 한 번만 분석·검증하고 잘못된 조합을 즉시 차단했다. 이후 미들웨어는 요청마다 재분석하지 않고 캐시된 클래스 정보만 사용해 데이터를 매핑한다.

Flow — 시작 1회 분석 → 매 요청 재사용



Result

- 잘못된 데코레이터 조합을 배포 전 서버 시작 시점에 차단 — 런타임 침묵 버그 제거
- 요청마다 재분석을 없애 런타임 경로를 단순화, 관심사 분리 원칙을 런타임까지 일관 확장

Redis ZSET으로 설계한 실시간 참여도 랭킹

Problem

Plum의 차별점은 수강생 참여도를 실시간으로 보이는 것이다. 모든 참가자가 랭킹을 실시간으로 조회하는 상황이라, 점수가 바뀔 때마다 순위를 빠르게 계산해 방송해야 했다. 단순 카운팅으로는 세 가지가 막혔다. 같은 점수일 때 누구를 위에 둘지(동점 처리), 점수를 도배하는 남용, 그리고 조회마다 전체를 정렬하는 부담이다. "정렬·동점·남용을 실시간 방송이 가능한 비용 안에서 어떻게 풀 것인가"가 문제였다.

Solving

1 — 정렬과 통계를 분리한 저장 구조

WHY 실시간 랭킹 조회가 빈번 · **HOW** 정렬용 ZSET · 통계용 Hash 이원화

모든 참가자가 랭킹을 수시로 조회하므로, 순위 응답이 빨라야 했다. 점수만 담아 정렬에 최적화된 Redis Sorted Set(ZSET)으로 랭킹을 관리하고, 발언·투표 같은 참가자별 상세 통계는 별도 Hash에 담아 역할을 나눴다. 통계 카운트는 pipeline으로, 원자적으로 일괄 갱신했다.

2 — 점수 획득 시점으로 동점을 가르기

WHY 같은 점수면 먼저 달성한 사람이 위 · **HOW** 점수에 획득 시점을 소수점으로 인코딩

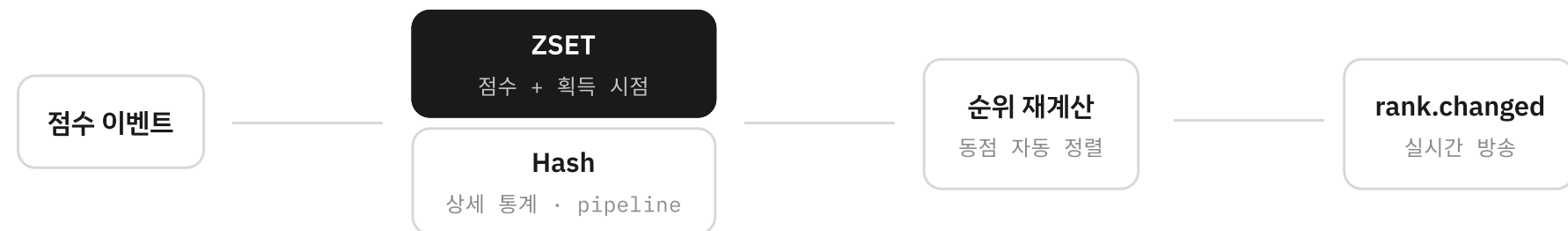
같은 점수일 때 먼저 도달한 참가자를 상위에 두려면, 조회 때마다 달성 시각을 따로 비교해 정렬하는 비용이 든다. 이를 없애기 위해 점수 점수에 점수 획득 시점을 소수점으로 인코딩해 ZSET score에 합산했다. 점수가 같아도 먼저 획득한 참가자가 우선하도록 순위가 자연히 갈려, 별도 비교 로직 없이 ZSET 정렬만으로 동점이 해결된다.

3 — 남용을 막는 단계적 페널티

WHY 점수 도배가 랭킹을 왜곡 · **HOW** 경고 누적에 따라 차감 후 초기화

점수를 반복 도배하는 남용을 막기 위해 단계적 페널티를 뒀다. 1·2차까지는 점수를 차감하는 경고로 대응하고, 패널티가 과도하게 누적되면 해당 참가자의 점수를 초기화해 남용 이득을 제거했다. 차감과 초기화를 나눠, 실수는 완화하되 반복 남용은 강하게 억제했다.

Score Model



tie-break — 동점 시 점수 획득이 빠른 참가자가 상위 · 별도 비교 로직 없음

Result

- 실시간 랭킹 조회를 ZSET 정렬로 처리하고, 동점은 획득 시점 인코딩으로 별도 비교 없이 해결
- 정렬(ZSET)과 통계(Hash)를 분리해 각 접근 패턴에 최적화
- 단계적 페널티로 점수 남용을 억제해 랭킹 신뢰성 확보

3분 내 요약 · 보안 · CPU 제약을 동시에

Problem

처음 주어진 요구는 "강의가 끝나면 빠르게 요약을 제공한다"라는 모호한 문장뿐이었다. 사용자가 강의 직후 복습한다는 점에 근거해 '종료 후 3분 내 요약 제공'을 기준으로 정의하자 3분 내 제공, 민감한 음성 데이터의 외부 유출 방지, 고가의 외부 API·GPU 없이 제한된 CPU 자원 내 처리라는 세 조건이 충돌했다

Solving

1 — 온프레미스 STT로 보안과 비용을 함께 해결

WHY 외부 전송 보안 위험 + 토큰 비용 · HOW 서버 내부 Faster-whisper 구동

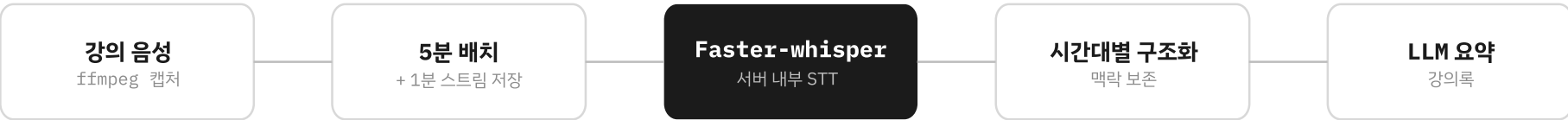
외부 API 대신 서버 내부에서 Faster-whisper를 직접 구동해 보안 위험과 토큰 비용을 제거했다. CPU 기반 인식의 정확도 저하는 LLM의 문맥 추론으로 보완했다.

2 — 5분 배치 + 1분 스트림 이중 구조, 실험으로 확정

WHY 짧게 끊으면 맥락 왜곡, 길면 지연 폭증 · HOW 60초~10분 반복 테스트

긴 음성을 5분 배치로 백그라운드에서 변환하고, 유실 방지를 위해 1분 스트림으로 다시 쪼개 저장했다. 배치 주기를 변수로 두고 주기별 변환 속도와 요약 품질을 함께 측정해, 두 기준을 모두 만족하는 5분을 최적값으로 확정했다.

Pipeline



실험 배치 주기 60초 → 10분 반복 테스트, 변환 속도·요약 품질 동시 측정 → 5분 확정

Result

- 강의 종료 후 3분 내 요약 제공 목표를 안정적으로 달성
- 메인 Node.js 프로세스 부하 최소화
- 모호한 요구를 측정 가능한 기준으로 바꾸고 실험으로 검증

Redis Pub/Sub 소켓 이벤트 순서 보장

Problem

다대다 실시간 강의에서 소켓 이벤트를 여러 백엔드 인스턴스가 나눠 처리하도록 Redis Pub/Sub을 소켓 이벤트 브로커로 썼다. 그런데 같은 사용자의 이벤트가 서로 다른 인스턴스로 분산되면서, 도착 순서가 발행 순서와 뒤바뀌는 문제가 생겼다. 실시간 투표·채팅처럼 순서가 곧 의미인 이벤트에서는 뒤섞임이 치명적이었다.

Solving

원인 — 이벤트 순서를 보장하는 주체가 없음

HOW 인스턴스 간 경합 분석

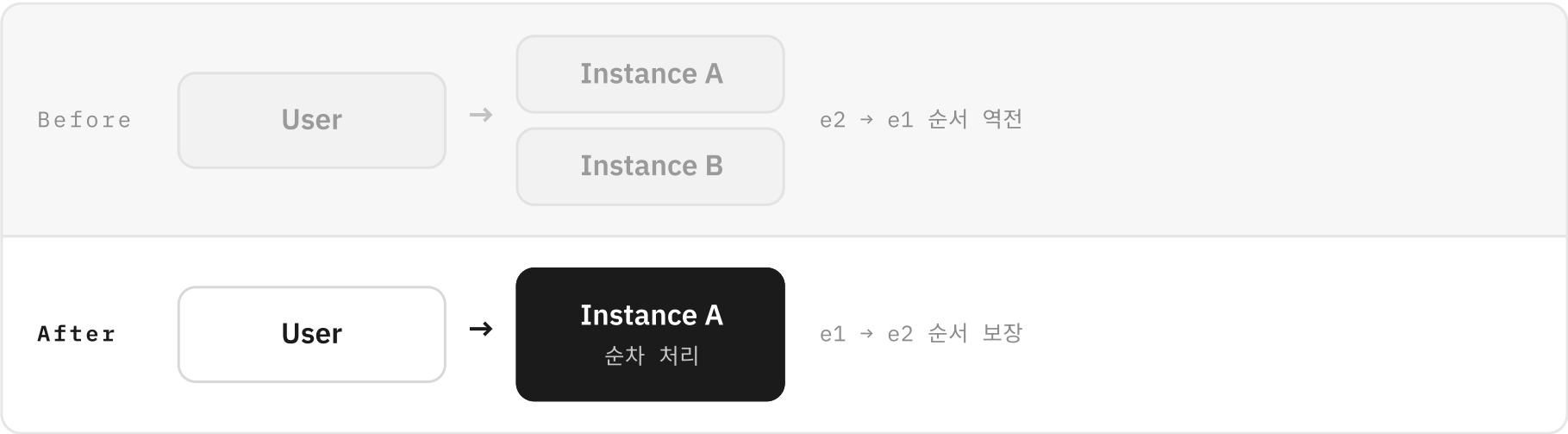
한 사용자의 연속 이벤트가 매번 다른 인스턴스로 흩어지니, 각 인스턴스가 독립적으로 처리하는 순간 전역 순서가 깨졌다. 브로커 계층 어디에도 "이 사용자의 이벤트는 순서대로"를 보장하는 주체가 없었던 것이 근본 원인이었다.

해결 — 연결 고정으로 동시성을 구조적으로 차단

WHY 사후 정렬보다 애초에 경합을 없앴 · HOW 사용자-인스턴스 연결 고정

이벤트를 나중에 정렬하는 대신, 한 사용자의 소켓을 특정 인스턴스에 고정해 연속 이벤트가 항상 같은 곳에서 순차 처리되게 했다. 순서 경합 자체가 발생하지 않도록 구조를 바꿔 동시성 문제를 원천 차단했다.

Before / After



Result

- 실시간 투표·채팅 이벤트의 순서 뒤섞임을 구조적으로 제거
- 사후 정렬 로직 없이 경합 자체를 없애, 스케일아웃 환경에서도 일관된 순서 유지

끊김과 이탈을 구분할 수 없는 재접속 문제

Problem

다자간 실시간 강의에서 한 참가자의 연결이 끊기면, 서버는 소켓이 끊겼다는 사실만 알 뿐 그게 어떤 상황인지 구분할 수 없었다. 잠깐 인터넷이 끊긴 건지, F5로 새로고침한 건지, 브라우저를 완전히 닫고 나간 건지가 서버 관점에서 모두 똑같이 "소켓 종료"로 보였다. 이 구분이 없으니 돌아올 사람을 곧바로 내보내 재접속 시 화면이 비거나, 끊긴 동안 바뀐 방 상태가 반영되지 않는 정합성 문제가 생겼다.

Solving

1 — 끊김을 즉시 이탈로 처리하지 않는 유예 구간

WHY 재접속과 완전 이탈은 서버에서 구분되지 않음 · **HOW** 끊김 후 유예 시간을 두고 상태 보존

소켓이 끊겼다고 바로 참가자를 제거하는 대신, 재접속 허용 유예 시간을 뒀다. 유예안에 돌아오면 재접속으로 간주해 기존 상태를 유지하고, 시간이 지나도록 돌아오지 않으면 그때 완전 이탈로 처리했다. 서버가 알 수 없는 "끊김의 의도"를 시간이라는 기준으로 대신 판별한 것이다.

2 — 재접속 시 현재 방 상태를 복원

WHY 끊긴 동안 방 상태가 계속 변함 · **HOW** 재입장 시 전체 상태·미디어 정보 재전달

재접속한 참가자에게는 현재 방의 참가자 전원 상태와 미디어 정보를 다시 전달해, 끊긴 동안 바뀐 상황이 화면에 즉시 반영되도록 했다. 재접속 시 Mediasoup 미디어 데이터를 초기화하고 다시 맞춰, 화면이 비거나 어긋나는 문제를 해소했다.

State



판별 — 서버가 알 수 없는 끊김의 의도를 유예 시간으로 대신 구분

Result

- 서버가 구분할 수 없는 끊김·이탈을 유예 시간 기준으로 판별해 재접속을 안정적으로 처리
- 재접속 시 현재 방 상태를 복원해 화면 공백·상태 불일치 해소
- 실시간 강의의 비정상 연결 상황을 옻지 케이스 중심으로 안정화

흩어진 요청 검증을 라이브러리로 표준화

Problem

1년간 기능을 쌓는 동안 컨트롤러 6종 각자 요청 payload를 파싱하고 검증했다. 비슷한 검증 코드가 컨트롤러마다 반복됐고, 새 검증을 추가하거나 방식을 맞추려면 매번 다른 컨트롤러를 열어 확인해야 했다. 검증 책임이 컨트롤러에 흩어져 있어, 처리 방식을 일관되게 유지하는 비용이 계속 늘어나는 게 문제였다.

Solving

1 — 왜 express-cargo를 도입했나

WHY 직접 만든 라이브러리를 실서비스에서 검증 · **HOW** 기성 대안과 교차 비교로 선택 정당성 확인

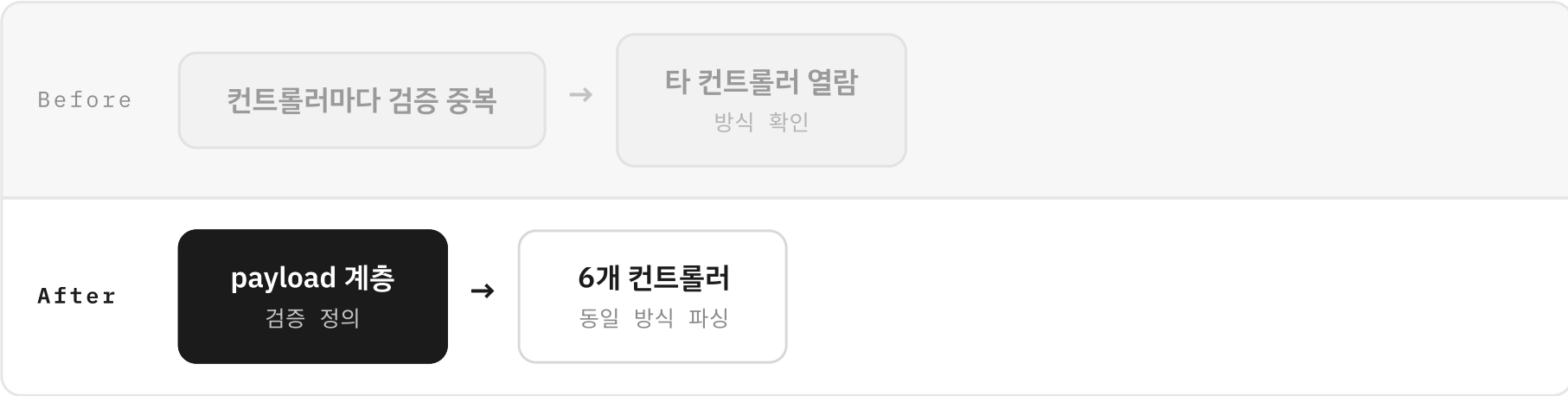
반복되는 검증 코드를 표준화할 시점에, 직접 만든 express-cargo를 실서비스에 적용해 설계를 검증해보려는 의도가 있었다. 다만 그 선택이 타당한지 기성 대안과 교차 비교했다. class-validator는 검증에 class-transformer 등 부가 라이브러리를 조합해야 했고, zod는 검증을 통과시키며 파싱 결과를 새 객체로 복제해 반환하는 방식이라 타입 검증만 필요한 상황에는 구조적으로 맞지 않았다. express-cargo가 이 환경에서 불리하지 않음을 확인하고 도입을 결정했다.

2 — 어떻게 표준화했나

WHY 검증 책임을 컨트롤러 밖으로 · **HOW** payload 계층 신설 후 전 컨트롤러 전환

요청 payload 타입을 types/payload/*로 별도 계층에 정의하고, 6개 컨트롤러의 파싱·검증을 express-cargo 기반의 동일한 방식으로 전환했다. 검증 로직이 컨트롤러에서 payload 정의로 옮겨가, 컨트롤러는 처리에만 집중하고 검증 규칙은 한곳에서 관리된다.

Before / After



Result

- 컨트롤러 6종의 요청 파싱·검증을 타입 안전한 단일 방식으로 통일
- 검증 책임을 컨트롤러에서 payload 계층으로 분리해, 규칙을 한곳에서 관리
- 직접 만든 오픈소스(express-cargo)를 실서비스에 적용해 설계를 검증한 사례

하나의 잘못된 데이터가 전체 수집을 막는 문제

Problem

외부 공공 API에서 행사 데이터를 정기 수집하는 배치를 운영하던 중, 원천 데이터에 잘못된 값이나 우리 서비스에 필요한 필드가 빠진 항목이 섞여 있었다. 이런 항목을 파싱하다 오류가 나면 해당 항목만 실패하는 게 아니라 **그 배치의 데이터 전체가 등록되지 않았다**. 소스가 서울 문화행사에 더해 관광공사 Tour까지 둘로 늘면서, 한 소스의 문제 데이터가 전체 수집을 멈추는 위험이 실제로 드러났다.

Solving

1 — 왜 두 번째 소스를 붙일 수 있었나

WHY 서비스 데이터 확보 · HOW 수집 로직을 함수 단위로 구조화해 재사용

처음엔 서울 문화행사 소스만 수집했지만, 이후 관광공사 Tour 데이터를 함께 써야 하는 상황이 생겼다. 첫 수집 로직을 함수 단위로 구조화해둔 덕에 그 구조를 재사용해 Tour 소스를 붙일 수 있었고, 원천이 둘로 늘며 서비스가 다루는 행사 데이터가 더 풍부해졌다. 대신 소스가 늘어난 만큼 한 소스의 장애가 미치는 범위도 함께 커졌다.

2 — 어떻게 장애를 격리했나

WHY 한 항목의 오류가 전체 등록을 막지 않도록 · HOW 항목 단위 try-catch로 오류 데이터 스킵

기존에는 잘못된 항목 하나에서 난 오류가 그 배치 전체의 등록을 막았다. 데이터를 항목 단위로 try-catch로 감싸, 오류가 난 항목은 건너뛰고 정상 항목은 그대로 등록되도록 바꿨다. 한 소스·한 항목의 실패가 다른 데이터 수집까지 멈추지 않게 에러를 격리한 것이다.

Before / After



Result

- 오류 항목을 격리해, 잘못된 데이터 하나가 전체 수집을 멈추지 않도록 개선
- 수집 로직을 함수 단위로 구조화해 관광공사 Tour 소스로 재사용 확장, 서비스 행사 데이터 확보
- 소스가 늘어난 배치를 장애에 견디는 방향으로 정리한 운영 관점의 개선

던지고 끝나지 않는 푸시 알림 시스템 설계

Problem

푸시 알림은 유저를 다시 불러오는 가장 직접적인 통로지만, 잘못 다루면 역효과가 크다. 플랫폼마다 프로토콜이 파편화돼 있고, 무분별한 발송은 앱 삭제를 부르며, 단말기에 도달하면 사라지는 휘발성 탓에 내역 관리도 안 된다. "메시지를 보낸다"가 아니라 **발송·필터링·수신 보장·영속성이 맞물린 시스템**으로 설계해야 하는 문제였다. 이 전 구간을 앱·서버·웹에 걸쳐 안드로이드 기준 제로베이스로 담당했다.

Solving

1 — 전송 통로 단일화

WHY 소규모 팀에서 플랫폼별 운영은 낭비 · **HOW** FCM 추상화로 OS 분기 위임

iOS·Android를 각각 다루는 대신 FCM으로 전송을 단일화했다. 서버는 한 번의 호출만 하고 OS별 전달은 Firebase에 위임해, APNs 프로토콜·인증서 관리 부담을 덜고 전송 로직의 일관성을 확보했다.

2 — 언제·누구에게 보낼지의 트리거 설계

WHY 무분별한 알림은 앱 삭제를 유발 · **HOW** 도메인 로직에 발송 조건을 심음

본인 상호작용은 대상에서 제외하고, '좋아요'는 이전 상태와 대조해 신규 발생 시에만 발송하고 해제 시에는 보내지 않았다. 알림 페이지에 딥링크를 실어, 알림을 누르면 해당 콘텐츠로 바로 진입하도록 재방문 동선을 좁혔다.

3 — 포그라운드 수신 보장

WHY 앱 사용 중 실시간 알림이 무시됨 · **HOW** 수신 메시지를 가로채 로컬 알림으로 강제 표시

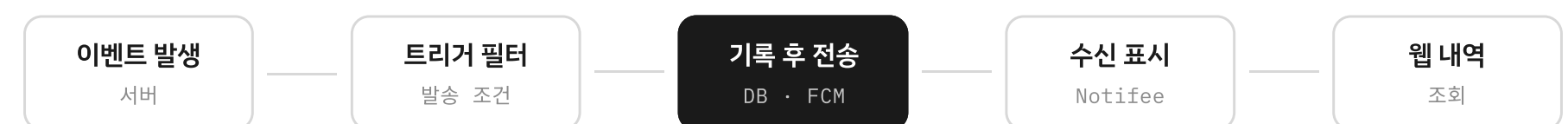
안드로이드는 포그라운드에서 FCM 메시지를 받아도 시스템이 팝업을 띄우지 않고 무시한다. 수신 메시지를 앱에서 가로채 Notifee로 시스템 알림을 강제로 띄우고, 8.0+에서 필수인 알림 채널을 높은 중요도로 정의해 어떤 상태에서도 노출되게 했다.

4 — 영속성과 토큰 정리

WHY 휘발성 알림의 내역 관리 + 전송 성공률 · **HOW** 기록 후 전송 + 실패 토큰 자동 제거

발송 직전 알림을 DB에 먼저 기록해 유실을 막고 앱 내 알림함 조회를 가능하게 했다. 멀티캐스트 전송 결과에서 실패한 토큰(앱 삭제 등)은 즉시 DB에서 제거해, 서버가 항상 유효한 토큰만 유지하도록 했다.

Pipeline



범위 — 앱(RN) · 서버(Express) · 웹(Next.js) 전 구간 단독 · 안드로이드 기준

Result

- 인프라 통합·도메인 필터링·수신 보장·영속성이 맞물린 알림 시스템을 앱·서버·웹 전 구간 단독 설계
- 안드로이드 포그라운드 수신 제약을 Notifee로 극복해 상태 무관 알림 보장
- 기록 후 전송과 토큰 정리로 알림 유실 방지 및 전송 성공률 유지

Windows에서만 실패하는 안드로이드 빌드

Problem

React Native 0.76 New Architecture 적용 후, macOS와 달리 Windows 팀원들만 안드로이드 빌드가 반복 실패했다. 옵션을 끄면 빌드는 됐지만, 성능 향상과 네이티브 모듈 통합이라는 도입 목적이 사라진다. Windows 팀원들의 안드로이드 빌드가 막힌 상황에서, 회피 대신 근본 원인을 추적하기로 했다.

Solving

1 — 원인 규명: CMake 경로 파싱 오류

WHY 설정 회피로는 도입 목적이 사라짐 · **HOW** CXX1429 빌드 로그 분석

CMake 빌드 로그에서 Invalid character escape '\U' 에러를 발견했다. Windows 경로 구분자 백슬래시가 유니코드 이스케이프로 오인식된 것으로, RN의 application.cmake가 Windows 경로를 처리하지 못하는 프레임워크 자체 결함임을 규명했다.

2 — 해결: 이슈 추적 후 버전 마이그레이션

WHY 이미 고쳐진 문제인지 먼저 확인 · **HOW** GitHub Issue · 릴리스 노트 전수 조사

같은 문제 사례를 조사해 이미 수정 PR이 머지된 것을 확인하고, 패치 버전 0.76.3으로 업그레이드했다. 캐시 충돌 방지를 위해 환경을 초기화하고 팀 전체에 공지해 개발 환경을 일치시켰다.

Flow



root cause – Invalid character escape '\U' · Windows 경로 구분자가 유니코드 이스케이프로 오인식

Result

- 설정 회피가 아닌 빌드 시스템·네이티브 레이어 분석으로 근본 해결, 팀 전체 개발 환경 일치
- 이후 팀 내 React Native 환경 설정과 트러블슈팅을 주도

플랫폼 제한 안에서 정확한 키워드 추출

Problem

문서 관계 그래프를 만들자, **모든 키워드를 링크로 연결해 그래프가 과밀해지는** 현상이 먼저 관찰됐다. 노이즈를 줄이려면 정확한 키워드 선별이 필요했지만, Forge의 백그라운드 함수 실행 시간 제한(30초)과 외부 API를 막는 보안 정책이 있었다. 또한 node 환경 NLP 라이브러리는 런타임 호환성 문제로 빌드 장애까지 일으켰다. "제한된 자원 안에서 어떻게 정확한 키워드만 남길 것인가"가 풀어야 할 문제였다.

Solving

1 — 왜 두 트랙을 나란히 두었는가

WHY 호환성·성능 제약 + 인스턴스 결제 제약 동시 대응 · **HOW** 두 방식을 각각 측정해 병렬 제공

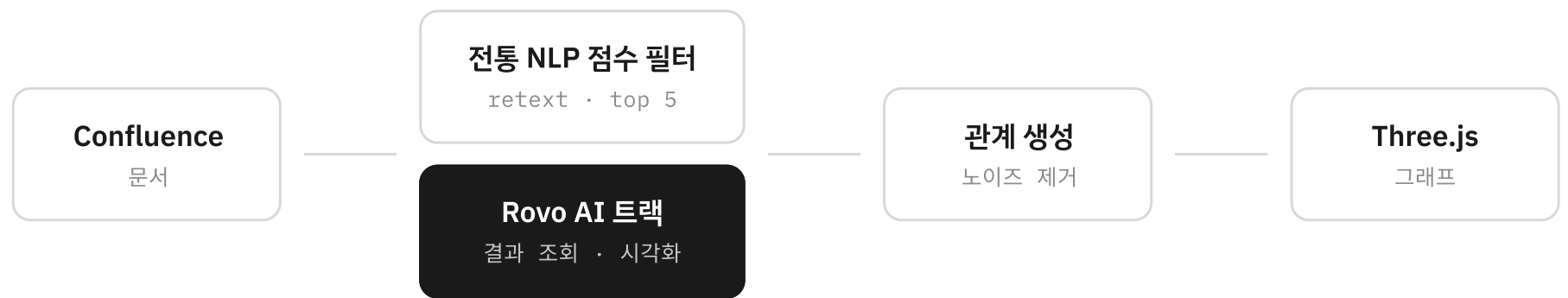
전통 NLP(retext)만 쓰면 문맥 파악이 약하고, AI(Rovo)만 쓰면 실행 시간 관리가 어려웠다. 게다가 Rovo는 해당 Confluence가 Rovo를 결제한 환경에서만 동작해, Rovo 하나만 택하면 미결제 환경에서는 키워드 추출 자체가 불가능했다. 그래서 두 방식을 각각 측정해, 어디서든 동작하는 retext 기반 추출과 문맥을 잡는 Rovo 기반 추출을 독립된 두 트랙으로 나란히 제공하는 이중 트랙으로 설계했다.

2 — 각 트랙에서 어떻게 구현했는가

WHY 실행 시간·노이즈 동시 축소 · **HOW** retext 상위 5개 랭킹 · Rovo 결과 조회 리졸버 신설

전통 NLP 필터에서는 키워드를 score·빈도 순으로 정렬해 상위 5개만 채택하고, 값 추출 로직을 어근으로 좁혀 불필요한 의존성을 제거했다. Rovo 트랙에서는 Rovo Agent가 사용할 리졸버를 신설해 일관된 키워드를 반환하도록 조정하고, 그 결과를 그래프 링크로 시각화하는 소비 계층을 구현했다.

Dual Track



기여 경계 — 두 트랙은 독립 제공 · 미결제 환경은 retext 트랙만 동작

Result

- 상위 5개 제한으로 링크·연산량을 줄여, 플랫폼 제한 안에서 처리 가능한 문서량 임계치를 확대
- 미결제 환경에서도 동작하는 retext와 문맥을 잡는 Rovo를 병렬 제공해, 환경과 관계없이 키워드 추출 보장
- 단순 연결이 아닌 핵심 주제 중심의 의미 있는 관계 그래프 시각화 구현